

Arrays and Strings in C#

1. Arrays in C#

✓ Description

An **array** is a collection of **same type of elements stored in continuous memory locations**.

👉 Arrays store multiple values in a single variable.

2. Syntax

```
dataType[] arrayName = new dataType[size];
```

3. Use of Arrays

- Store multiple values in one variable
- Easy data management
- Used in loops and data processing
- Efficient memory usage

4. FULL EXAMPLE (Array)

```
using System;
```

```
class Program
```

```
{  
    static void Main()  
    {  
        int[] numbers = new int[5];  
  
        numbers[0] = 10;  
        numbers[1] = 20;  
        numbers[2] = 30;  
        numbers[3] = 40;  
        numbers[4] = 50;  
  
        Console.WriteLine("Array Elements:");  
  
        for (int i = 0; i < numbers.Length; i++)  
        {  
            Console.WriteLine(numbers[i]);  
        }  
    }  
}
```

5. Types of Arrays

- Single-dimensional array
- Multi-dimensional array
- Jagged array

6. FULL EXAMPLE (2D Array)

```

using System;

class Program
{
    static void Main()
    {
        int[,] matrix = {
            {1, 2},
            {3, 4}
        };

        for (int i = 0; i < 2; i++)
        {
            for (int j = 0; j < 2; j++)
            {
                Console.Write(matrix[i, j] + " ");
            }
            Console.WriteLine();
        }
    }
}

```

7. Strings in C#

✓ Description

A **string** is a sequence of characters used to store text data.

👉 In C#, string is an object of the System.String class.

8. Syntax

```
string name = "Hello";
```

9. Use of Strings

- Store text data
- Used in input/output
- Used in file handling and processing
- Widely used in real-world applications

10. FULL EXAMPLE (String)

```

using System;

class Program
{
    static void Main()
    {
        string name = "C# Programming";

        Console.WriteLine("Original: " + name);
        Console.WriteLine("Uppercase: " + name.ToUpper());
    }
}

```

```

        Console.WriteLine("Lowercase: " + name.ToLower());
        Console.WriteLine("Length: " + name.Length);
    }
}

```

11. Common String Operations

- Length
- ToUpper()
- ToLower()
- Substring()
- Replace()

Jagged Array in C#

1. Description

A **jagged array** is an array of **arrays**, where each row can have a **different number of columns**.

👉 Unlike a 2D array (fixed size), jagged arrays are **irregular in structure**.

Example idea:

- Row 1 → 2 elements
- Row 2 → 4 elements
- Row 3 → 3 elements

2. Syntax

```
dataType[][] arrayName = new dataType[rowSize][];
```

Then each row is initialized separately:

```
arrayName[0] = new dataType[size];
```

3. Use of Jagged Arrays

- Used when data is **not uniform**
- Saves memory (no unused space)
- Useful in real-world data like:
 - Student marks (different subjects per student)
 - Departments with different number of employees

4. FULL EXAMPLE

```
using System;
```

```

class Program
{
    static void Main()
    {
        // Jagged array declaration
        int[][] jagged = new int[3][];

        // Initializing rows with different sizes
        jagged[0] = new int[] { 1, 2 };
        jagged[1] = new int[] { 3, 4, 5, 6 };
    }
}

```

```

jagged[2] = new int[] { 7, 8, 9 };

Console.WriteLine("Jagged Array Elements:");

for (int i = 0; i < jagged.Length; i++)
{
    for (int j = 0; j < jagged[i].Length; j++)
    {
        Console.Write(jagged[i][j] + " ");
    }
    Console.WriteLine();
}
}
}

```

Properties in C#

1. Description

A **property** in C# is a **member of a class that provides a flexible way to read, write, or compute private fields**.

👉 It acts as a **bridge between private data (fields) and outside code**.

- Used to control access to class variables
- Provides **data encapsulation**
- Replaces traditional getter and setter methods

2. Syntax

```

class ClassName
{
    private int field;

    public int PropertyName
    {
        get
        {
            return field;
        }
        set
        {
            field = value;
        }
    }
}

```

3. Use of Properties

- Protect data using encapsulation

- Control how values are assigned or retrieved
- Add validation logic
- Improve code readability
- Common in object-oriented applications

4. FULL EXAMPLE (Basic Property)

using System;

```
class Student
{
    private int id;
    private string name;
```

```
    // Property for ID
```

```
    public int Id
```

```
    {
        get
        {
            return id;
        }
        set
        {
            id = value;
        }
    }
```

```
    // Property for Name
```

```
    public string Name
```

```
    {
        get
        {
            return name;
        }
        set
        {
            name = value;
        }
    }
```

```
    public void Display()
```

```
    {
        Console.WriteLine("ID: " + Id);
        Console.WriteLine("Name: " + Name);
    }
}
```

```
class Program
```

```
{
```

```

static void Main()
{
    Student s = new Student();

    s.Id = 101;
    s.Name = "Ram";

    s.Display();
}
}

```

5. AUTO-IMPLEMENTED PROPERTY

✓ Description

C# also provides **short syntax** when no extra logic is needed.

✓ Syntax

```
public int Id { get; set; }
```

✓ FULL EXAMPLE

```
using System;
```

```

class Student
{
    public int Id { get; set; }
    public string Name { get; set; }

    public void Display()
    {
        Console.WriteLine(Id + " " + Name);
    }
}

```

```

class Program
{
    static void Main()
    {
        Student s = new Student();

        s.Id = 1;
        s.Name = "Sita";

        s.Display();
    }
}

```

Fields in C#

1. Description

A **field** in C# is a **variable declared inside a class or struct** that is used to store data.

👉 Fields represent the **state (data) of an object**.

- Can be public or private
- Usually kept private for encapsulation
- Accessed using properties or methods

2. Syntax

```
class ClassName
{
    dataType fieldName;
}
```

3. Use of Fields

- Store object data (state)
- Hold values inside a class
- Work as internal storage for properties
- Used in object-oriented programming to define attributes of objects

4. FULL EXAMPLE

```
using System;

class Student
{
    // Fields (data members)
    private int id;
    private string name;

    // Method to assign values
    public void SetData(int i, string n)
    {
        id = i;
        name = n;
    }

    // Method to display values
    public void Display()
    {
        Console.WriteLine("ID: " + id);
        Console.WriteLine("Name: " + name);
    }
}

class Program
{
    static void Main()
```

```

{
    Student s = new Student();

    s.SetData(101, "Ram");
    s.Display();
}
}

```

5. Types of Fields (Important)

✓ Instance Field

- Belongs to object
- Each object has its own copy

✓ Static Field

- Shared among all objects
- Declared using static keyword

6. FULL EXAMPLE (Static Field)

using System;

```

class Student
{
    public static string schoolName = "ABC School";
}

```

```

class Program
{
    static void Main()
    {
        Console.WriteLine(Student.schoolName);
    }
}

```

Indexers in C#

1. Description

An **indexer** in C# allows an object to be accessed like an **array using square brackets []**.

👉 It provides a way to access class data using index values instead of method calls.

- Works like “smart arrays”
- Used to access private data in a class
- Helps in cleaner and more readable code

2. Syntax

```

class ClassName
{
    dataType[] arrayName = new dataType[size];

    public dataType this[int index]

```

```

{
    get
    {
        return arrayName[index];
    }
    set
    {
        arrayName[index] = value;
    }
}
}

```

3. Use of Indexers

- Provides array-like access to objects
- Improves code readability
- Encapsulates internal data structure
- Used in collections and data structures

4. FULL EXAMPLE

using System;

```

class StudentCollection
{
    private string[] names = new string[3];

    // Indexer
    public string this[int index]
    {
        get
        {
            return names[index];
        }
        set
        {
            names[index] = value;
        }
    }
}

class Program
{
    static void Main()
    {
        StudentCollection s = new StudentCollection();

        // Setting values using indexer
        s[0] = "Ram";
        s[1] = "Sita";
    }
}

```

```

s[2] = "Hari";

// Getting values using indexer
for (int i = 0; i < 3; i++)
{
    Console.WriteLine(s[i]);
}
}
}

```

Constructors in C# (Syntax, Description, Use & Full Examples)

A **constructor** is a special method used to initialize objects automatically when a class is instantiated. It has the same name as the class and no return type.

1. Default Constructor

✓ Description

A constructor that takes **no parameters** and assigns default values or performs basic initialization.

✓ Syntax

```

class ClassName
{
    ClassName()
    {
        // initialization code
    }
}

```

✓ Use

- Initializes object with default values
- Automatically called when object is created

✓ Full Example

```

using System;

class Student
{
    public int id;
    public string name;

    // Default Constructor
    public Student()
    {
        id = 1;
        name = "Default Student";
    }

    public void Display()
    {
        Console.WriteLine(id + " " + name);
    }
}

```

```

    }
}

class Program
{
    static void Main()
    {
        Student s1 = new Student();
        s1.Display();
    }
}

```

2. Parameterized Constructor

✓ Description

A constructor that accepts parameters to initialize object with **user-defined values**.

✓ Syntax

```

class ClassName
{
    ClassName(type1 param1, type2 param2)
    {
        // assignment
    }
}

```

✓ Use

- Assign custom values at object creation
- Reduces need for separate setter methods

✓ Full Example

```

using System;

class Student
{
    public int id;
    public string name;

    // Parameterized Constructor
    public Student(int i, string n)
    {
        id = i;
        name = n;
    }

    public void Display()
    {
        Console.WriteLine(id + " " + name);
    }
}

```

```

class Program
{
    static void Main()
    {
        Student s1 = new Student(101, "Ram");
        Student s2 = new Student(102, "Sita");

        s1.Display();
        s2.Display();
    }
}

```

3. Copy Constructor

✓ Description

A constructor that creates a **new object by copying values from an existing object**.

✓ Syntax

```

class ClassName
{
    ClassName(ClassName obj)
    {
        // copy values
    }
}

```

✓ Use

- Create duplicate objects
- Useful in cloning object data

✓ Full Example

```
using System;
```

```

class Student
{
    public int id;
    public string name;

    public Student(int i, string n)
    {
        id = i;
        name = n;
    }

    // Copy Constructor
    public Student(Student s)
    {
        id = s.id;
        name = s.name;
    }
}

```

```

    public void Display()
    {
        Console.WriteLine(id + " " + name);
    }
}

class Program
{
    static void Main()
    {
        Student s1 = new Student(101, "Hari");
        Student s2 = new Student(s1); // copying

        s1.Display();
        s2.Display();
    }
}

```

4. Static Constructor

✓ Description

A constructor used to initialize **static members of a class**. It runs only once automatically.

✓ Syntax

```

class ClassName
{
    static ClassName()
    {
        // static initialization
    }
}

```

✓ Use

- Initialize static variables
- Executes only once before any object creation

✓ Full Example

```

using System;

class Student
{
    public static string schoolName;

    // Static Constructor
    static Student()
    {
        schoolName = "ABC School";
        Console.WriteLine("Static Constructor Called");
    }

    public void Display()

```

```

    {
        Console.WriteLine("School: " + schoolName);
    }
}

```

```

class Program
{
    static void Main()
    {
        Student s1 = new Student();
        s1.Display();

        Student s2 = new Student();
        s2.Display();
    }
}

```

5. Constructor Overloading

✓ Description

Multiple constructors with different parameter lists in the same class.

✓ Syntax

```

ClassName() { }
ClassName(int x) { }
ClassName(int x, string y) { }

```

✓ Use

- Flexibility in object creation
- Different ways to initialize object

✓ Full Example

```
using System;
```

```

class Student
{
    public int id;
    public string name;

    public Student()
    {
        id = 0;
        name = "Unknown";
    }

    public Student(int i)
    {
        id = i;
        name = "No Name";
    }
}

```

```

public Student(int i, string n)
{
    id = i;
    name = n;
}

public void Display()
{
    Console.WriteLine(id + " " + name);
}
}

```

```

class Program
{
    static void Main()
    {
        Student s1 = new Student();
        Student s2 = new Student(10);
        Student s3 = new Student(20, "Rita");

        s1.Display();
        s2.Display();
        s3.Display();
    }
}

```

Use of base Keyword in C#

The base keyword in C# is used to refer to the **parent (base) class** from a derived class. It is mainly used to access:

- Base class constructor
- Base class methods
- Base class fields/properties (if accessible)

✓ 1. Calling Base Class Constructor

Description

Used to call the constructor of the parent class from the child class.

Syntax

```
class Derived : Base
```

```

{
    Derived() : base()
    {
    }
}

```

Use

- Initialize base class data before derived class
- Ensures proper object creation order

Example

```

using System;

class Parent
{
    public Parent()
    {
        Console.WriteLine("Parent Constructor");
    }
}

class Child : Parent
{
    public Child() : base()
    {
        Console.WriteLine("Child Constructor");
    }
}

class Program
{
    static void Main()
    {
        Child obj = new Child();
    }
}

```

✓ 2. Calling Base Class Method

Description

Used when a derived class wants to call the method of its parent class.

◆ Syntax

```
base.MethodName();
```

◆ Use

- Access overridden parent behavior
- Extend functionality instead of replacing it

◆ Example

```

using System;

class Parent
{
    public void Show()
    {
        Console.WriteLine("Parent Show Method");
    }
}

class Child : Parent
{

```

```

    public void Show()
    {
        base.Show(); // calling parent method
        Console.WriteLine("Child Show Method");
    }
}

```

```

class Program
{
    static void Main()
    {
        Child obj = new Child();
        obj.Show();
    }
}

```

✓ 3. Accessing Base Class Members

◆ Description

Used to access base class fields or properties when they are hidden in derived class.

◆ Example

using System;

```

class Parent
{
    public string name = "Parent Class";
}

class Child : Parent
{
    public string name = "Child Class";

    public void Display()
    {
        Console.WriteLine(name);    // Child class field
        Console.WriteLine(base.name); // Parent class field
    }
}

```

```

class Program
{
    static void Main()
    {
        Child obj = new Child();
        obj.Display();
    }
}

```

Method Overriding vs Method Hiding in C#

Both concepts are used in **inheritance** when a derived class defines a method with the same name as a base class method.

1. Method Overriding

✓ Description

Method overriding occurs when a **derived class provides a new implementation of a base class method** using the override keyword. It supports **runtime polymorphism**.

✓ Conditions

- Base method must be marked virtual, abstract, or override
- Derived method must use override
- Method signature must be same

✓ Syntax

```
class Base
{
    public virtual void Show()
    {
        Console.WriteLine("Base Class Method");
    }
}

class Derived : Base
{
    public override void Show()
    {
        Console.WriteLine("Derived Class Method");
    }
}
```

✓ Full Example

```
using System;

class Parent
{
    public virtual void Display()
    {
        Console.WriteLine("Parent Display");
    }
}

class Child : Parent
{
    public override void Display()
    {
        Console.WriteLine("Child Display");
    }
}
```

```

    }

class Program
{
    static void Main()
    {
        Parent obj = new Child();
        obj.Display(); // Runtime polymorphism
    }
}

```

✓ Use

- Achieves **runtime polymorphism**
- Used when behavior must change in derived class

2. Method Hiding

✓ Description

Method hiding occurs when a derived class defines a method with the same name as base class method using the new keyword. It hides the base method.

✓ Syntax

```

class Base
{
    public void Show()
    {
        Console.WriteLine("Base Class Method");
    }
}

class Derived : Base
{
    public new void Show()
    {
        Console.WriteLine("Derived Class Method");
    }
}

```

✓ Full Example

using System;

```

class Parent
{
    public void Display()
    {
        Console.WriteLine("Parent Display");
    }
}

```

```

class Child : Parent
{
    public new void Display()
    {
        Console.WriteLine("Child Display");
    }
}

class Program
{
    static void Main()
    {
        Child obj = new Child();
        obj.Display();

        Parent obj2 = new Child();
        obj2.Display(); // Calls Parent method
    }
}

```

✓ Use

- Used when base method is not virtual
- Used to redefine behavior without polymorphism

Key Differences

Feature	Method Overriding	Method Hiding
Keyword	override	new
Polymorphism	Runtime	No runtime polymorphism
Base method	virtual/abstract	normal method
Binding	Dynamic	Static
Best use	flexible behavior	simple replacement

Applying Polymorphism in Code Extensibility (FULL ANSWER)

1. Description

Polymorphism allows a **single base class reference** to behave differently depending on the object it points to. It improves **code extensibility** because new classes can be added without changing existing code.

2. Syntax

```

class BaseClass
{
    public virtual void Show()
    {
        Console.WriteLine("Base class method");
    }
}

```

```
}  
}
```

```
class DerivedClass : BaseClass  
{  
    public override void Show()  
    {  
        Console.WriteLine("Derived class method");  
    }  
}
```

3. Use

- To achieve **runtime polymorphism**
- To make code **flexible and scalable**
- To allow **new classes without modifying existing logic**
- Common in real-world systems like payments, shapes, employees, etc.

4. FULL EXAMPLE (Complete Program)

using System;

// Base Class

```
class Shape  
{  
    public virtual void Draw()  
    {  
        Console.WriteLine("Drawing Shape");  
    }  
}
```

// Derived Class 1

```
class Circle : Shape  
{  
    public override void Draw()  
    {  
        Console.WriteLine("Drawing Circle");  
    }  
}
```

// Derived Class 2

```
class Rectangle : Shape  
{  
    public override void Draw()  
    {  
        Console.WriteLine("Drawing Rectangle");  
    }  
}
```

// Derived Class 3 (Extensibility)

```

class Triangle : Shape
{
    public override void Draw()
    {
        Console.WriteLine("Drawing Triangle");
    }
}

```

```

// Main Program
class Program
{
    static void Main()
    {
        Shape s;

        s = new Circle();
        s.Draw(); // Circle version

        s = new Rectangle();
        s.Draw(); // Rectangle version

        s = new Triangle();
        s.Draw(); // Triangle version
    }
}

```

Interface in C#

1. Description

An **interface** in C# is a contract that defines **what a class should do but not how it should do it**.

- It contains only method declarations (no implementation)
- It is used to achieve **full abstraction**
- A class can implement multiple interfaces

👉 Interface = 100% abstraction model

2. Syntax

```

interface IInterfaceName
{
    void Method1();
    void Method2();
}

```

Implementation:

```

class ClassName : IInterfaceName
{
    public void Method1()
    {
        // implementation
    }
}

```

```
public void Method2()
{
    // implementation
}
}
```

3. Use of Interface

- Achieve **multiple inheritance in C#**
- Provide **common structure for different classes**
- Improve **loose coupling and flexibility**
- Used in real-world systems like payment systems, logging, drivers, etc.

4. FULL EXAMPLE

✓ Step 1: Interface Definition

```
using System;
```

```
interface IAnimal
{
    void Sound();
}
```

✓ Step 2: Implementing Interface in Classes

```
class Dog : IAnimal
{
    public void Sound()
    {
        Console.WriteLine("Dog barks");
    }
}
```

```
class Cat : IAnimal
{
    public void Sound()
    {
        Console.WriteLine("Cat meows");
    }
}
```

✓ Step 3: Using Interface (Main Program)

```
class Program
{
    static void Main()
    {
        IAnimal a;

        a = new Dog();
    }
}
```

```

        a.Sound();

        a = new Cat();
        a.Sound();
    }
}

```

Abstract Class in C#

1. Description

An **abstract class** is a class that **cannot be instantiated (no object can be created)** and is used as a **base class for other classes**.

It can contain:

- Abstract methods (no body)
- Normal methods (with implementation)
- Fields, properties, constructors

👉 It is used when we want to provide **partial implementation + force derived classes to complete behavior**.

2. Syntax

```

abstract class ClassName
{
    public abstract void Method1(); // abstract method

    public void Method2()
    {
        // normal method
    }
}

```

3. Use of Abstract Class

- To achieve **partial abstraction**
- To define **common behavior for all derived classes**
- To enforce implementation in child classes
- Used in real-world frameworks and base designs (e.g., Shape, Employee systems)

4. FULL EXAMPLE

✓ Step 1: Abstract Class

```
using System;
```

```

abstract class Animal
{
    // Abstract method (no body)
    public abstract void Sound();

    // Normal method
    public void Sleep()
    {

```

```

        Console.WriteLine("Sleeping...");
    }
}

```

✓ Step 2: Derived Class 1

```

class Dog : Animal
{
    public override void Sound()
    {
        Console.WriteLine("Dog barks");
    }
}

```

✓ Step 3: Derived Class 2

```

class Cat : Animal
{
    public override void Sound()
    {
        Console.WriteLine("Cat meows");
    }
}

```

✓ Step 4: Main Program

```

class Program
{
    static void Main()
    {
        Animal a;

        a = new Dog();
        a.Sound();
        a.Sleep();

        a = new Cat();
        a.Sound();
        a.Sleep();
    }
}

```

Sealed Class in C#

1. Description

A **sealed class** in C# is a class that **cannot be inherited** (no child class can be created from it).

It is used to:

- Prevent inheritance
- Restrict class extension
- Ensure security and fixed implementation

👉 Once a class is declared sealed, it becomes the **final class in the hierarchy**.

2. Syntax

```
sealed class ClassName
{
    // members
}
```

3. Use of Sealed Class

- To prevent further inheritance
- To protect class behavior from modification
- Used when class implementation is complete and final
- Improves security and performance slightly (compiler optimization)

4. FULL EXAMPLE

✓ Step 1: Sealed Class Definition

```
using System;
```

```
sealed class Vehicle
{
    public void Run()
    {
        Console.WriteLine("Vehicle is running");
    }
}
```

Step 2: Trying to Inherit (Not Allowed)

```
// This will give compile-time error
```

```
// class Car : Vehicle
// {
// }
```

👉 A sealed class **cannot be inherited**

✓ Step 3: Using Sealed Class

```
class Program
{
    static void Main()
    {
        Vehicle v = new Vehicle();
        v.Run();
    }
}
```

Partial Class in C#

1. Description

A **partial class** in C# allows a single class definition to be **split into multiple files**. All parts are combined into one class at **compile time**.

👉 It is mainly used in large projects to:

- Organize code
- Separate logic
- Improve maintainability

2. Syntax

✓ Part 1 (File 1)

```
partial class ClassName
{
    // members
}
```

✓ Part 2 (File 2)

```
partial class ClassName
{
    // more members
}
```

3. Use of Partial Class

- Split large classes into multiple files
- Useful in **auto-generated code (e.g., Windows Forms, ASP.NET)**
- Improves readability and teamwork development
- Allows multiple developers to work on same class safely

4. FULL EXAMPLE

✓ File 1: Student_Info.cs

```
using System;
```

```
partial class Student
{
    public int id;
    public string name;

    public void GetData(int i, string n)
    {
        id = i;
        name = n;
    }
}
```

✓ File 2: Student_Display.cs

```
using System;
```

```
partial class Student
{
    public void Display()
    {
        Console.WriteLine("ID: " + id);
    }
}
```

```

        Console.WriteLine("Name: " + name);
    }
}

```

✓ Main Program

```

class Program
{
    static void Main()
    {
        Student s = new Student();
        s.GetData(101, "Ram");
        s.Display();
    }
}

```

Delegate in C#

1. Description

A **delegate** in C# is a type that represents a **reference to a method**.

👉 It is used to:

- Pass methods as parameters
- Call methods dynamically
- Achieve **event handling and callback mechanism**

Think of a delegate as a **function pointer (safe version in C#)**.

2. Syntax

✓ Step 1: Declare Delegate

```
delegate returnType DelegateName(parameterList);
```

✓ Step 2: Assign Method

```
DelegateName obj = MethodName;
```

✓ Step 3: Call Delegate

```
obj(parameters);
```

3. Use of Delegate

- Used for **event handling**
- Used in **callback methods**
- Enables **loose coupling between classes**
- Used in GUI programming and asynchronous operations

4. FULL EXAMPLE

✓ Step 1: Declare Delegate

```
using System;
```

```
// Delegate declaration
```

```
delegate void MyDelegate(string message);
```

✓ Step 2: Methods Matching Delegate

```

class Program
{
    static void ShowMessage(string msg)
    {
        Console.WriteLine("Message: " + msg);
    }

    static void ShowUpper(string msg)
    {
        Console.WriteLine("Upper: " + msg.ToUpper());
    }
}

```

✓ Step 3: Using Delegate

```

static void Main()
{
    MyDelegate del;

    del = ShowMessage;
    del("Hello Delegate");

    del = ShowUpper;
    del("hello delegate");
}
}

```

5. Multicast Delegate (Important Concept)

A delegate can hold **multiple methods at once**.

```

MyDelegate del = ShowMessage;
del += ShowUpper;

```

```

del("Hello");

```

👉 Both methods will execute.

Events in C#

1. Description

An **event** in C# is a mechanism used to provide **notification to other objects when something happens**.

👉 Events are based on **delegates** and are mainly used in **event-driven programming** (like button clicks, notifications, alarms).

- Publisher → raises the event
- Subscriber → responds to the event

👉 Example: Button click in GUI applications

2. Syntax

✓ Step 1: Delegate Declaration

```

delegate void MyEventHandler();

```

✓ Step 2: Event Declaration

```
class Publisher
{
    public event MyEventHandler MyEvent;
}
```

✓ Step 3: Raise Event

```
MyEvent();
```

3. Use of Events

- Used in GUI applications (button click, mouse events)
- Used in notifications and alerts
- Enables communication between objects
- Supports **event-driven programming model**

4. FULL EXAMPLE (Publisher–Subscriber Model)

✓ Step 1: Delegate Declaration

```
using System;
```

```
delegate void Notify();
```

✓ Step 2: Publisher Class (Event Source)

```
class Publisher
{
    public event Notify OnProcessCompleted;

    public void StartProcess()
    {
        Console.WriteLine("Process Started...");

        // Process logic completed
        Console.WriteLine("Process Completed");

        // Raise event
        OnProcessCompleted();
    }
}
```

✓ Step 3: Subscriber Class

```
class Subscriber
{
    public void ShowMessage()
    {
        Console.WriteLine("Subscriber: Process Completed Event Received");
    }
}
```

✓ Step 4: Main Program (Wiring Event)

```

class Program
{
    static void Main()
    {
        Publisher p = new Publisher();
        Subscriber s = new Subscriber();

        // Subscribe event
        p.OnProcessCompleted += s.ShowMessage;

        // Start process
        p.StartProcess();
    }
}

```

Collections in C#

1. Description

A **collection in C#** is a **group of related objects** stored and managed as a single unit.

Unlike arrays, collections are:

- Dynamic in size (can grow or shrink)
- Provide built-in methods for data handling

👉 Collections are part of **System.Collections** and **System.Collections.Generic** namespace.

2. Syntax

✓ ArrayList (Non-Generic Collection)

```
using System.Collections;
```

```
ArrayList list = new ArrayList();
```

✓ Generic Collection (List<T>)

```
using System.Collections.Generic;
```

```
List<int> list = new List<int>();
```

3. Use of Collections

- Store multiple objects efficiently
- Dynamic memory allocation
- Easy operations like add, remove, search
- Used in databases, applications, and data handling systems

4. FULL EXAMPLE (ArrayList – Non Generic Collection)

```
using System;
```

```
using System.Collections;
```

```

class Program
{
    static void Main()

```

```

{
    ArrayList list = new ArrayList();

    // Adding items
    list.Add(10);
    list.Add(20);
    list.Add(30);

    Console.WriteLine("ArrayList Elements:");

    foreach (object item in list)
    {
        Console.WriteLine(item);
    }

    // Removing item
    list.Remove(20);

    Console.WriteLine("After Removal:");

    foreach (object item in list)
    {
        Console.WriteLine(item);
    }
}

```

5. FULL EXAMPLE (Generic Collection – List<T>)

```

using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        List<int> numbers = new List<int>();

        numbers.Add(100);
        numbers.Add(200);
        numbers.Add(300);

        Console.WriteLine("List Elements:");

        foreach (int num in numbers)
        {
            Console.WriteLine(num);
        }
    }
}

```

```

        numbers.Remove(200);

        Console.WriteLine("After Removal:");

        foreach (int num in numbers)
        {
            Console.WriteLine(num);
        }
    }
}

```

Generics in C#

1. Description

Generics in C# allow you to define **classes, methods, and collections with a placeholder type**.

👉 Instead of specifying a fixed data type, you use a **type parameter (T)**.

Benefits:

- Type safety at compile time
- Code reusability
- Better performance (no boxing/unboxing)

2. Syntax

✓ Generic Class

```

class ClassName<T>
{
    T data;
}

```

✓ Generic Method

```

void MethodName<T>(T value)
{
}

```

✓ Generic Collection

```

List<T> list = new List<T>();

```

3. Use of Generics

- To create reusable and type-safe code
- To avoid duplicate code for different data types
- To improve performance
- Common in collections like List<T>, Stack<T>, Queue<T>

4. FULL EXAMPLE (Generic Class)

```

using System;

```

```

class Box<T>
{
    private T value;
}

```

```

    public void SetValue(T v)
    {
        value = v;
    }

    public T GetValue()
    {
        return value;
    }
}

class Program
{
    static void Main()
    {
        // Integer type
        Box<int> intBox = new Box<int>();
        intBox.SetValue(100);
        Console.WriteLine(intBox.GetValue());

        // String type
        Box<string> strBox = new Box<string>();
        strBox.SetValue("Hello Generics");
        Console.WriteLine(strBox.GetValue());
    }
}

```

5. FULL EXAMPLE (Generic Method)

using System;

```

class Program
{
    static void Show<T>(T value)
    {
        Console.WriteLine(value);
    }

    static void Main()
    {
        Show<int>(10);
        Show<string>("C# Generics");
        Show<double>(99.5);
    }
}

```

File I/O in C#

1. Description

File Input/Output (File I/O) in C# is used to **create, read, write, and manipulate files** stored on a disk.

It allows programs to:

- Store data permanently
- Read existing files
- Modify or update file content

👉 File handling is part of **System.IO** namespace.

2. Syntax (Basic)

```
using System.IO;
```

3. Common File I/O Classes

Class	Purpose
FileStream	Read/write byte data
StreamReader	Read text from file
StreamWriter	Write text to file
MemoryStream	Work with memory data
StringReader	Read from string
StringWriter	Write to string

4. Use of File I/O

- Store user data permanently
- Read configuration files
- Write logs and reports
- Data processing applications
- File-based database systems

5. FULL EXAMPLE 1: Writing to a File (StreamWriter)

```
using System;
```

```
using System.IO;
```

```
class Program
{
    static void Main()
    {
        StreamWriter writer = new StreamWriter("sample.txt");

        writer.WriteLine("Hello C# File I/O");
        writer.WriteLine("This is file writing example");

        writer.Close();

        Console.WriteLine("Data written successfully");
    }
}
```

6. FULL EXAMPLE 2: Reading from a File (StreamReader)

```
using System;
using System.IO;

class Program
{
    static void Main()
    {
        StreamReader reader = new StreamReader("sample.txt");

        string line;

        while ((line = reader.ReadLine()) != null)
        {
            Console.WriteLine(line);
        }

        reader.Close();
    }
}
```

7. FULL EXAMPLE 3: FileStream (Byte-Level Operation)

```
using System;
using System.IO;

class Program
{
    static void Main()
    {
        FileStream fs = new FileStream("data.txt", FileMode.Create);

        byte[] data = System.Text.Encoding.UTF8.GetBytes("Hello FileStream");

        fs.Write(data, 0, data.Length);

        fs.Close();

        Console.WriteLine("File created using FileStream");
    }
}
```

Try Statements and Exceptions in C#

1. Description

In C#, **exception handling** is used to handle **runtime errors** so that the program does not crash unexpectedly.

👉 The try-catch-finally block is used to manage exceptions safely.

- **try** → code that may cause an error
- **catch** → handles the error
- **finally** → always executes (error or not)

2. Syntax

```
try
{
    // risky code
}
catch(Exception e)
{
    // handle error
}
finally
{
    // always executed
}
```

3. Use of Exception Handling

- Prevent program crash
- Handle runtime errors gracefully
- Improve application reliability
- Debugging and error logging
- Used in file handling, database, and user input systems

4. FULL EXAMPLE 1: Basic Try-Catch

using System;

```
class Program
{
    static void Main()
    {
        try
        {
            int a = 10;
            int b = 0;

            int result = a / b; // error (divide by zero)

            Console.WriteLine(result);
        }
        catch(Exception e)
        {
            Console.WriteLine("Error occurred: " + e.Message);
        }
    }
}
```

5. FULL EXAMPLE 2: Try-Catch-Finally

using System;

```
class Program
{
    static void Main()
    {
        try
        {
            int[] arr = {1, 2, 3};

            Console.WriteLine(arr[5]); // error
        }
        catch(Exception e)
        {
            Console.WriteLine("Exception handled: " + e.Message);
        }
        finally
        {
            Console.WriteLine("Finally block always executes");
        }
    }
}
```

6. FULL EXAMPLE 3: Multiple Catch Blocks

using System;

```
class Program
{
    static void Main()
    {
        try
        {
            string str = null;
            Console.WriteLine(str.Length);
        }
        catch(NullReferenceException e)
        {
            Console.WriteLine("Null error: " + e.Message);
        }
        catch(Exception e)
        {
            Console.WriteLine("General error: " + e.Message);
        }
    }
}
```

Attributes in C# (Attribute Classes, Named & Positional Parameters)

1. Description

An **attribute** in C# is a special tag used to **add metadata (extra information) to code elements** like classes, methods, properties, or assemblies.

👉 Attributes do not change program logic directly but provide information to:

- Compiler
- Runtime
- Tools and frameworks

2. Syntax

```
[AttributeName]  
class ClassName  
{  
}
```

3. Attribute Classes

Attributes in C# are created using **classes derived from System.Attribute.**

✓ Syntax

```
using System;
```

```
class MyAttribute : Attribute  
{  
}
```

✓ Use

- Define custom metadata
- Apply rules or extra information to code elements

✓ Example

```
using System;
```

```
[AttributeUsage(AttributeTargets.Class)]  
class InfoAttribute : Attribute  
{  
    public string Description;  
  
    public InfoAttribute(string desc)  
    {  
        Description = desc;  
    }  
}
```

```
[Info("This is Student class")]  
class Student  
{  
}
```

4. Named Parameters in Attributes

✓ Description

Named parameters allow you to **assign values to specific properties of an attribute** using property names.

✓ Syntax

```
[AttributeName(Property = value)]
```

✓ Example

using System;

```
class InfoAttribute : Attribute
{
    public string Author { get; set; }
}
```

```
[Info(Author = "Nile")]
class Book
{
}
```

✓ Use

- Improves readability
- Allows optional values in attributes
- Makes code flexible

5. Positional Parameters in Attributes

✓ Description

Positional parameters are **passed through constructor of the attribute class**.

✓ Syntax

```
[AttributeName(value1, value2)]
```

✓ Example

using System;

```
class InfoAttribute : Attribute
{
    public string Name;

    public InfoAttribute(string name)
    {
        Name = name;
    }
}
```

```
[Info("Computer Science")]
class Department
```

```
{  
}
```

6. Difference Between Named & Positional Parameters

Feature	Positional	Named
Passed through Constructor		Properties
Order matters	Yes	No
Syntax	[Attr("value")]	[Attr(Name="value")]
Flexibility	Less	More

7. Use of Attributes

- Add metadata to program elements
- Used in frameworks like ASP.NET
- Helps reflection and runtime analysis
- Used for validation and configuration

Attribute Targets in C#

1. Description

Attribute Targets define **where an attribute can be applied** in a C# program.

👉 They control whether an attribute can be used on:

- Class
- Method
- Property
- Field
- Assembly
- Parameter, etc.

This is controlled using the **AttributeUsage** class.

2. Syntax

```
[AttributeUsage(AttributeTargets.Class)]  
class MyAttribute : Attribute  
{  
}
```

3. Common Attribute Targets

Target	Meaning
Class	Applied to class
Method	Applied to method
Property	Applied to property
Field	Applied to variable
Constructor	Applied to constructor
Assembly	Applied to entire assembly
Parameter	Applied to method parameter

4. FULL EXAMPLE (Single Target)

✓ Applying Attribute Only to Class

using System;

```
[AttributeUsage(AttributeTargets.Class)]
```

```
class InfoAttribute : Attribute
```

```
{  
    public string Message;
```

```
    public InfoAttribute(string msg)
```

```
    {  
        Message = msg;  
    }  
}
```

```
[Info("This is Student class")]
```

```
class Student
```

```
{  
}
```

5. FULL EXAMPLE (Multiple Targets)

✓ Attribute used on Class and Method

using System;

```
[AttributeUsage(AttributeTargets.Class | AttributeTargets.Method)]
```

```
class InfoAttribute : Attribute
```

```
{  
    public string Message;
```

```
    public InfoAttribute(string msg)
```

```
    {  
        Message = msg;  
    }  
}
```

```
[Info("Applied on Class")]
```

```
class Student
```

```
{  
    [Info("Applied on Method")]  
    public void Show()  
    {  
        Console.WriteLine("Hello");  
    }  
}
```

Specifying Multiple Attributes in C#

1. Description

In C#, you can apply **more than one attribute** to a single class, method, or other program element.

👉 This is called **Specifying Multiple Attributes**.

Each attribute adds **extra metadata** to the same program element.

2. Syntax

✓ Multiple attributes written separately

```
[Attribute1]
[Attribute2]
class ClassName
{
}
```

✓ Multiple attributes in one line (less common)

```
[Attribute1, Attribute2]
class ClassName
{
}
```

3. Use of Multiple Attributes

- Add multiple metadata rules to a single element
- Used in frameworks like ASP.NET and validation systems
- Helps define multiple behaviors for same class/method
- Improves flexibility and configuration control

4. FULL EXAMPLE

✓ Step 1: Define Attributes

```
using System;
```

```
[AttributeUsage(AttributeTargets.Class | AttributeTargets.Method)]
class AuthorAttribute : Attribute
{
    public string Name;

    public AuthorAttribute(string name)
    {
        Name = name;
    }
}
```

```
class VersionAttribute : Attribute
{
    public string Number;

    public VersionAttribute(string num)
    {
        Number = num;
    }
}
```

```
}  
}
```

✓ Step 2: Applying Multiple Attributes

```
[Author("Nile")]  
[Version("1.0")]  
class Student  
{  
    [Author("Admin")]  
    [Version("1.1")]  
    public void Show()  
    {  
        Console.WriteLine("Student details");  
    }  
}
```

✓ Step 3: Main Program

```
class Program  
{  
    static void Main()  
    {  
        Student s = new Student();  
        s.Show();  
    }  
}
```

C# Unit–1 Practical Assignment Questions

Section A: Basics (Fields, Properties, Constructors)

1.

Create a class Student with:

- fields: id, name, marks
- properties for each field
- parameterized constructor
- method Display()

👉 Create 3 objects and display data.

2.

Write a program to demonstrate:

- default constructor
- parameterized constructor
- copy constructor

Use class Employee.

Section B: Arrays & Strings

3.

Write a program to:

- store 10 integers in an array
- find sum, average, max and min

4.

Write a program to:

- input a string
- convert to uppercase, lowercase
- find length
- reverse string

5.

Create a **jagged array** for students where:

- each student has different number of subjects
- display all marks

Section C: OOP Concepts

6.

Create class Shape with method Area().

Derive:

- Circle
- Rectangle

Use method overriding.

7.

Create a program to demonstrate:

- method overloading
 - method overriding
- in a single program.

8.

Create a program using:

- inheritance
 - base keyword
- Call base class constructor and method.

Section D: Abstract Class & Interface

9.

Create an abstract class Animal with:

- abstract method Sound()
- concrete method Sleep()

Implement in Dog and Cat.

10.

Create an interface IShape with method Draw().

Implement in:

- Circle
- Square
- Triangle

Section E: Polymorphism

11.

Create a program demonstrating **runtime polymorphism**:

- Base class Vehicle
- Derived classes Car, Bike, Bus
- Method Run()

12.

Write a program to show **code extensibility using polymorphism**:

- Add new class without modifying existing code

Section F: Collections & Generics

13.

Write a program using:

- ArrayList
- Add, remove, display elements

14.

Write a program using List<int>:

- add 10 numbers
- remove even numbers
- display remaining

15.

Create a generic class Box<T>:

- store int, string, double values
- display stored values

Section G: File I/O

16.

Write a program to:

- write student data into a file
- read and display file content

17.

Write a program using:

- StreamWriter
 - StreamReader
- to store and retrieve employee details.

Section H: Exception Handling

18.

Write a program using:

- try-catch-finally
- Handle divide by zero exception.

19.

Write a program to handle:

- `NullPointerException`
- `IndexOutOfRangeException`

Section I: Indexers

20.

Create a class `BookStore` using indexer:

- store 5 book names
- access books using index []

Section J: Delegates & Events

21.

Create a delegate program:

- perform Add, Subtract, Multiply using delegate

22.

Create a publisher-subscriber event system:

- Publisher raises event
- Subscriber handles event

Section K: Attributes

23.

Create custom attribute `InfoAttribute`:

- apply on class and method
- display metadata using reflection

24.

Demonstrate:

- positional parameters
- named parameters
in attributes

Final Combined Assignment (Important)

25.

Create a mini project:

👉 “Student Management System” using:

- Class & Object
- Constructor
- Properties
- Array/List
- File I/O
- Exception Handling
- Menu-driven program